

Three things that changed today.

-
- 01 Qwen's next architecture appears in a preview: half the hidden size, 512 experts, 62% less KV cache.
 - 02 Those linear-attention layers keep a fixed state in FP32. Quantizing it frees 69% of that memory.
 - 03 A checkpoint can pass every FP16 check and behave differently after INT8. Certification does not survive quantization.

Qwen's next architecture appears in a preview: half the hidden size, 512 experts, 62% less KV cache.

- hidden 5120, 64 layers, 4 KV heads, dense FFN (Qwen3.8-27B)
- + hidden 2560, 48 layers, 2 KV heads, 512-expert MoE (Flash-Next)

WHY IT MATTERS

Fewer KV heads and 12 full-attention layers change what one GPU can hold, before any kernel work.

WHO SHOULD CARE

Anyone sizing GPUs for long-context serving, or comparing this preview against Qwen3.8-27B.

The two published configs, diffed.

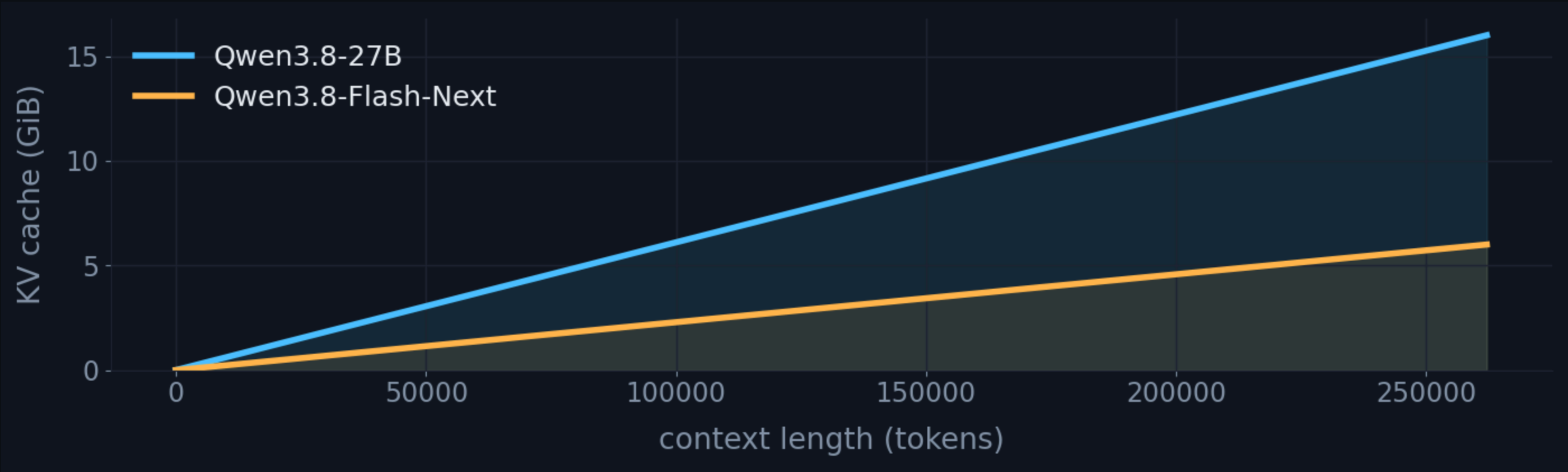
```
import json, pathlib
H = pathlib.Path(__file__).parent
cfg = lambda n: json.loads((H / n).read_text())["text_config"]
old, new = cfg("config_qwen38_27b.json"), cfg("config_flash_next.json")

def kv_per_token(c):          # only full-attention layers hold a growing cache
    full = c["layer_types"].count("full_attention")
    return full * 2 * c["num_key_value_heads"] * c["head_dim"] * 2    # K+V, bfloat16
...
print(f"KV cache per token    {100*(1-kv_per_token(new)/kv_per_token(old)):.1f}% smaller")
```

Qwen3.8-27B	layers= 64	full_attn= 16	kv_heads=4	kv/token= 64 KiB
Qwen3.8-Flash-Next	layers= 48	full_attn= 12	kv_heads=2	kv/token= 24 KiB
KV cache per token	62.5% smaller			
experts	512 total, 10 active (2.0% per token)			

24 KiB

KV cache per token, against 64 KiB for Qwen3.8-27B — 62.5% less



Those linear-attention layers keep a fixed state in FP32. Quantizing it frees 69% of that memory.

- recurrent state stored in FP32 – 32 bits per value
- + critical channels kept high, the rest INT8 – 9.9 bits average

WHY IT MATTERS

Uniform INT8 already hurts reasoning accuracy and INT4 destroys it. Mixed precision is what makes the saving usable.

WHO SHOULD CARE

Anyone serving Gated DeltaNet or Kimi Delta Attention models at batch, where state memory competes with weights.

State size from the real config: 32 bits against 9.9.

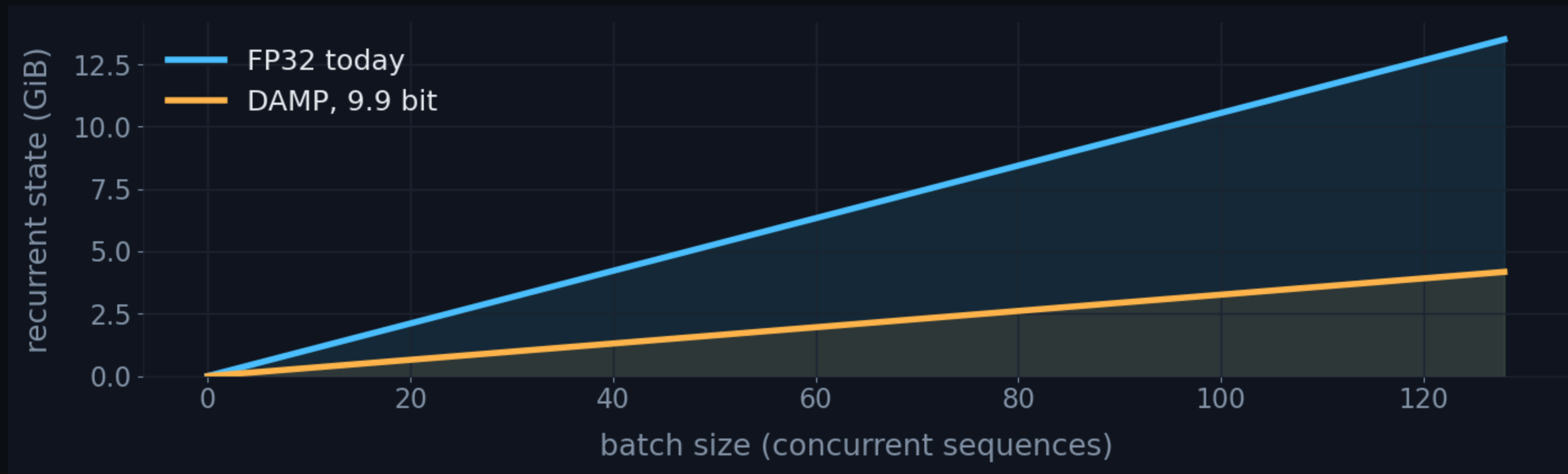
```
linear = c["layer_types"].count("linear_attention")           # layers with a fixed-size state
per_layer = c["linear_num_value_heads"] * c["linear_key_head_dim"] * c["linear_value_head_dim"]
values = linear * per_layer                                   # state values per sequence
BATCH = 64

for label, bits in (("FP32, as stored today", 32.0), ("DAMP, 9.9 bit/value", 9.9)):
    per_seq = values * bits / 8
    print(f"{label:24} {per_seq/2**20:7.1f} MiB/seq    {per_seq*BATCH/2**30:5.2f} GiB at batch {BATCH}")
```

FP32, as stored today	108.0 MiB/seq	6.75 GiB at batch 64
DAMP, 9.9 bit/value	33.4 MiB/seq	2.09 GiB at batch 64
state values per seq	28,311,552	(36 linear-attention layers)
storage reduction	69.1%	<- the paper's headline number, from its own bit budget

6.75 → 2.09 GiB

recurrent-state memory at batch 64, computed from the Flash-Next config



A checkpoint can pass every FP16 check and behave differently after INT8. Certification does not survive quantization.

- certify the FP16 checkpoint, ship the INT8 build
- + certify the configuration you actually deploy

WHY IT MATTERS

If you validate the FP16 checkpoint and ship the quantized one, you certified a model you did not deploy.

WHO SHOULD CARE

Anyone shipping quantized weights to edge devices, or accepting third-party checkpoints on trust.

Same output at full precision. Different output after INT8.

```
w = rng.normal(0, 0.05, 4096).astype(np.float64) # one weight row
x = rng.normal(0, 1.00, 4096).astype(np.float64) # the validation input
q, s = int8(w)

r = w / s - np rint(w / s) # distance to nearest INT8 code
push = np.sign(r) * (0.5 - np.abs(r) + 1e-3) * s # nudge every weight just past its boundary
push -= (push @ x) / (x @ x) * x # ...orthogonal to x: FP64 output unchanged
```

weights moved by	0.406% of $\max w $	
full precision $w @ x$	-0.906887	-> -0.906887 (identical to 6 dp)
after INT8 $w @ x$	-0.936712	-> -3.157366
INT8 codes changed	2232/4096	

0% → 85.02%

friend-foe inversion in their backdoored translation model, after quantization

TAKEAWAY

Two papers, same day, compressing the same states: one frees 4.7 GiB at batch 64, the other shows you may not know what you deployed.

[FULL CODE IN DESCRIPTION](#)

[A NEW DIFF EVERY WEEKDAY](#)